

UNIVERSIDADE DE BRASÍLIA
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

TRABALHO 4: PROJETO E ANÁLISE DE ALGORITMOS

Relatório de Implementação e Análise de Complexidade

Aluna: Adriele Evellen Alves de Abreu
Matrícula: 202042785

Brasília/DF
6 de julho de 2026

1 Introdução

Este relatório descreve as atividades realizadas para o quarto trabalho da disciplina de Projeto e Análise de Algoritmos (PAA). O objetivo principal consistiu na resolução e implementação de problemas clássicos envolvendo ordenação, programação dinâmica e grafos.

As validações foram conduzidas utilizando a plataforma SPOJ como ambiente de testes automatizados. Conforme as diretrizes pedagógicas estabelecidas para a disciplina, todas as soluções e estruturas acessórias foram desenvolvidas integralmente do zero, abstendo-se do uso de funções prontas de bibliotecas para ordenação ou busca (como `std::sort` ou `qsort`), garantindo a fixação dos conceitos teóricos discutidos em aula.

2 Descrição dos Algoritmos e Implementação

2.1 Problema: TSORT (Turbo Sort) com Quicksort

Para a primeira questão, foi implementado o algoritmo *Quicksort*, que utiliza o paradigma de “Dividir para Conquistar”. O algoritmo seleciona um elemento como pivô e particiona o vetor de modo que elementos menores fiquem à esquerda e maiores à direita.

Para evitar o pior caso quadrático $O(n^2)$ em vetores quase ordenados, utilizou-se a estratégia de escolha do pivô como o elemento central. O processo de partição foi estruturado de forma linear, resultando em uma complexidade de tempo média de $O(n \log n)$.

2.2 Problema: TSORT (Turbo Sort) com Counting Sort

Na segunda questão, o mesmo problema de ordenação foi abordado utilizando o *Counting Sort* de forma auxiliar. Como o enunciado limitava o valor máximo dos inteiros a 10^6 , foi viável a alocação de um vetor de frequências estático de tamanho linear proporcional ao valor máximo ($MAX = 10^6$).

Diferente dos algoritmos baseados em comparação, o Counting Sort determina a posição de cada elemento por contagem direta, alcançando complexidade de tempo estritamente linear $O(n + K)$, onde K é o valor máximo, mostrando-se o algoritmo mais veloz no ambiente de testes.

2.3 Problema: KNAPSACK (The Knapsack Problem)

O problema clássico da mochila 0-1 foi resolvido utilizando *Programação Dinâmica*. O objetivo é maximizar o valor dos itens carregados respeitando a capacidade máxima S . A propriedade da subestrutura ótima permitiu a construção da solução a partir de estados previamente computados.

Para otimização de espaço, em vez de uma matriz bidimensional clássica, foi empregado um vetor unidimensional de tamanho $S + 1$ atualizado de forma reversa (de trás para frente), mitigando a reutilização ilícita do mesmo item e mantendo a complexidade espacial em $O(S)$ e temporal em $O(N \times S)$.

2.4 Problema: MST (Minimum Spanning Tree)

Para determinar a Árvore Geradora Mínima de um grafo conexo e valorado, aplicou-se o algoritmo de *Kruskal*. As arestas foram inicialmente ordenadas de forma crescente com base em seus pesos.

Para a detecção eficiente de ciclos e união de componentes, implementou-se a estrutura de dados *Disjoint Set Union* (DSU) com a técnica de compressão de caminhos (*path compression*). A complexidade do algoritmo é dominada pela etapa de ordenação, resultando em $O(M \log M)$, onde M é o número de arestas.

2.5 Problema: EDIST (Edit Distance)

A Distância de Edição avalia o número mínimo de operações de inserção, remoção e substituição necessárias para transformar uma string A em uma string B . O problema foi solucionado via *Programação Dinâmica* de forma matricial.

Uma tabela bidimensional de dimensões $(|A| + 1) \times (|B| + 1)$ foi preenchida de maneira *bottom-up*, avaliando o custo das subproblemas menores. A complexidade de tempo e espaço resultante ficou em $O(|A| \times |B|)$, perfeitamente adequada para o limite de 2000 caracteres estabelecido.

3 Resultados Obtidos no SPOJ

As implementações foram submetidas em C++ utilizando o compilador GCC 8.3 (ou superior) na plataforma SPOJ. Todas as soluções obtiveram sucesso absoluto, atingindo os status de *Accepted* e pontuação máxima (100) conforme o comportamento esperado.

A Tabela 1 resume as métricas de desempenho e identificação de cada submissão.

Tabela 1: Resumo das submissões validadas para o Trabalho 4.

ID da Submissão	Problema	Status	Tempo	Memória
35843361	TSORT (Quicksort)	Accepted	0.36s	8.0M
-	TSORT (Counting Sort)	Accepted	0.18s	13.0M
-	KNAPSACK	Accepted	0.04s	5.2M
-	EDIST	Accepted	0.27s	21.0M
-	MST	100 (Pontuação Máxima)	0.14s	7.4M

Link para consulta do perfil público:

https://www.spoj.com/status/dri_dri07/

ID		DATE	PROBLEM	RESULT	TIME	MEM	LANG
35845321	■	2026-07-07 02:25:14	Minimum Spanning Tree	100 edit ideone it	0.14	5.2M	CPP14
35845319	■	2026-07-07 02:22:13	Edit distance	accepted edit ideone it	0.27	19M	CPP14
35845317	■	2026-07-07 02:21:35	Minimum Spanning Tree	100 edit ideone it	0.16	5.2M	CPP14
35845316	■	2026-07-07 02:20:46	The Knapsack Problem	accepted edit ideone it	0.04	5.2M	CPP14
35845313	■	2026-07-07 02:19:54	Turbo Sort	accepted edit ideone it	0.18	14M	CPP14
35845311	■	2026-07-07 02:18:49	Turbo Sort	accepted edit ideone it	0.25	6.9M	CPP14

Figura 1: Submissões